

StaySync — Guía Completa de Tests Unitarios

Framework: JUnit 5 (Jupiter) · Mockito 5 · AssertJ · Spring Test Utilities

Extensión base: `@ExtendWith(MockitoExtension.class)` — modo estricto en todos los archivos

Cobertura mínima: 60% de líneas por módulo (JaCoCo 0.8.12)

Fecha de auditoría: 2026-06-25 · Total: **11 archivos** — **92 tests unitarios**

1. Resumen Global

#	Archivo	Microservicio	Clase bajo test	Tests	Mocks clave
1	ReservaServiceTest	reservas	ReservaService	21	ReservaRepository, ReservaEventPublisher, RestTemplate
2	HabitacionServiceTest	habitaciones	HabitacionService	12	HabitacionRepository, TipoHabitacionRepository, HabitacionEventPublisher
3	ServicioServiceTest	servicios	ServicioService	10	ServicioRepository, SolicitudServicioRepository, RabbitTemplate
4	StripeServiceTest	pagos	StripeService	8	StripeGateway, PagoRepository, RabbitTemplate, RestTemplate, StripeConfig
5	UsuarioServiceTest	usuarios	UsuarioService	8	UsuarioRepository, PasswordEncoder, UsuarioMapper
6	NotificacionServiceTest	notificaciones	NotificacionService	8	NotificacionRepository, PlantillaNotificacionRepository, JavaMailSender
7	AuthServiceTest	usuarios	AuthService	6	AuthenticationManager, JwtService, RefreshTokenRepository, UsuarioRepository
8	JwtServiceTest	bff	JwtService	6	Ninguno (clase pura — solo ReflectionTestUtils para @Value)
9	PagoServiceTest	pagos	PagoService	5	PagoRepository, RabbitTemplate
10	CanalOtaServiceTest	ota	CanalOtaService	5	CanalOtaRepository, ReservaOtaRepository
11	HabitacionEventListenerTest	ota	HabitacionEventListener	3	CanalOtaService
TOTAL				92	

2. ReservaServiceTest — 21 tests

Dependencias mockeadas:

`@Mock ReservaRepository` — sin BD real
`@Mock ReservaEventPublisher` — sin RabbitMQ real
`@Mock RestTemplate` — sin llamadas a habitaciones-service
`@InjectMocks ReservaService`

Fix aplicado en setUp(): `fechaEntrada = LocalDate.now().plusDays(5)` en lugar de `plusDays(1)`. Con `plusDays(1)` el test fallaba en la tarde (hora Chile) porque `horasRestantes < 24` → `CancelacionRestringidaException` inesperada.

GRUPO: CREAR()

1	debeCrearReservaConPrecioReal	RestTemplate devuelve {precioPorNoche: 90.0, capacidad: 3} → precio calculado correctamente, eventPublisher.publicar("reserva.confirmada") llamado
2	debeCrearReservaConPrecioPorDefecto	RestTemplate lanza RuntimeException ("Connection refused") → servicio usa precio de fallback (100.00), reserva guardada igual
3	debeLanzarExcepcionCapacidadExcedida	request.numHuespedes=5, habitación.capacidad=3 → IllegalArgumentException "capacidad máxima", save() nunca llamado
4	debeLanzarExcepcionConConflicto	existeConflicto()==true → HabitacionNoDisponibleException, save() nunca llamado
5	debeGuardarHuespedesAdicionales	request incluye 1 huésped adicional → save() llamado 2 veces (1ª reserva, 2ª huéspedes)
6	debeLanzarExcepcionFechasInvalidas	fechaSalida == fechaEntrada (no posterior) → IllegalArgumentException "posterior"

GRUPO: OBTENERPORID() / LISTAR()

7	debeRetornarReserva	findById(1L) → Optional.of(reservaBase) → response.getCodigo() == "RES-ABCD1234"
8	debeLanzarExcepcionNoExiste	findById(99L) → Optional.empty() → ReservaNotFoundException
9	debeListarReservas	findAll(pageable) → PageImpl con 1 reserva → result.getTotalElements() == 1

GRUPO: CAMBIARESTADO()

10	debeCambiarEstadoPendienteAConfirmada	PENDIENTE → CONFIRMADA: estado cambiado + publicar("reserva.confirmada") verificado
11	debeCambiarEstadoConfirmadaACheckin	CONFIRMADA → CHECKIN: estado cambiado + publicar("reserva.checkin") verificado

12	debeLanzarExcepcionTransicionInvalida	CHECKOUT → CHECKIN (inválida) → TransicionEstadoInvalidaException
GRUPO: CANCELAR()		
13	debeCancelarReserva	CONFIRMADA + fechaEntrada en 5 días (>24h) → CANCELADA + publicar("reserva.cancelada")
14	debeLanzarExcepcionCancelacionFueraDePlazo	fechaEntrada = hoy (<24h) → CancelacionRestringidaException "24 horas", save() y publicar() nunca llamados
15	debeLanzarExcepcionSiYaCancelada	estado CANCELADA → TransicionEstadoInvalidaException (no se puede cancelar lo ya cancelado)
16	debeLanzarExcepcionSiYaCheckout	estado CHECKOUT → TransicionEstadoInvalidaException
GRUPO: AUTOTRANSICIONARACHECKIN() — SCHEDULER		
17	debeAutoTransicionarReservasConfirmadasDeHoy	findConfirmadasParaAutoCheckin(today) devuelve 1 reserva → CHECKIN + save() + publicar("reserva.checkin")
18	noDebeHacerNadaSinReservasParaHoy	findConfirmadasParaAutoCheckin → lista vacía → save() never, publicar() never
19	debeProcessarMultiplesReservasEnAutoCheckin	2 reservas CONFIRMADAS → save() times(2) + publicar("reserva.checkin") times(2)
GRUPO: GETRESERVASHOY()		
20	debeRetornarReservasDelDia	findByEstado(CONFIRMADA, today) + findByEstado(CHECKIN) → Map con pendientesCheckin[1] y pendientesCheckout[1]
21	debeRetornarListasVaciasConSinReservas	ambos findByEstado devuelven empty → pendientesCheckin y pendientesCheckout vacías

3. HabitacionServiceTest — 12 tests

HabitacionService

StaySync_Habitaciones-

habitaciones_v1.0/.../service/HabitacionServiceTest.java

```
@Mock HabitaciónRepository, @Mock TipoHabitacionRepository, @Mock AmenidadRepository,
@Mock HabitaciónEventPublisher
```

```
setUp(): tipo = TipoHabitacion("Estándar", capacidad=2, precio=80) · habitacion = Habitación("101", piso=1,
DISPONIBLE, precio=100)
```

CRUD BÁSICO

1	debeCrearHabitacion	existsByNumero("102")=false + findById(tipold)=tipo → save() llamado → response no null
2	debeLanzarExcepcionNumeroExistente	existsByNumero("101")=true → HabitacionNumeroExistenteException · save() never
3	debeRetornarHabitacion	findByIdAndActivaTrue(1L)=habitacion → response.getNumero()=="101"
4	debeLanzarExcepcionNoExiste	findByIdAndActivaTrue(99L)=empty → HabitacionNotFoundException
5	debeCambiarEstadoYPublicarEvento	cambiarEstado(1L, OCUPADA) → habitacion.estado==OCUPADA + publicarCambioEstado(1L, "DISPONIBLE", "OCUPADA") verificado con eq()
6	debeListarDisponibles	findByEstadoAndActivaTrue(DISPONIBLE) → lista de 1 → response[0].getEstado()==DISPONIBLE
7	debeDesactivarHabitacion	findByIdAndActivaTrue → habitacion.activa=false · save(habitacion) verificado
8	debeListarSoloActivas	findByActivaTrue() → [habitacion] → result.size()==1, numero=="101"
9	debeRetornarListaVacíaConSinActivas	findByActivaTrue() → [] → result.isEmpty()
10	debeLanzarExcepcionNumeroNoEncontrado	findByNumero("999")=empty → HabitacionNotFoundException

BUSCARDISPONIBLES() — FILTROS Y ORDENACIÓN

11	debeFiltrarPorCapacidadMinima	Devuelve habitacion(cap=2) y suite(cap=4); buscarDisponibles(capacidad=4) → solo suite[1] pasa el filtro
12	debeOrdenarPorPrecioDescendente	Devuelve habitacion(100) y penthouse(500); sort="precio_desc" → result[0].precio==500, result[1].precio==100

4. ServicioServiceTest — 10 tests

```
@Mock ServicioRepository, @Mock SolicitudServicioRepository, @Mock RabbitTemplate
```

```
setUp(): categoria=CategoriaServicio("Spa") · servicio=Servicio("Masaje 60min", precio=60, disponible=true, requiereReserva=true)
```

CATÁLOGO DE SERVICIOS

- | | | |
|---|------------------------------------|---|
| 1 | debeListarDisponibles | findByDisponibleTrue() → [servicio] → result[0].getNombre() == "Masaje 60min" |
| 2 | debeRetornarServicio | findById(1L)=servicio → response.getPrecio() == 60 |
| 3 | debeLanzarExcepcionNoExiste | findById(99L)=empty → ServicioNotFoundException |

SOLICITUDES DE SERVICIO

- | | | |
|----|--|---|
| 4 | debeCrearSolicitud | request(servicio=1, cantidad=2) → precio=60×2=120 → save() llamado → response.getPrecioTotal() == 120 |
| 5 | debeLanzarExcepcionServicioInexistente | findById(99L)=empty en crearSolicitud() → ServicioNotFoundException · solicitudRepository.save() never |
| 6 | debeListarSolicitudesPorReserva | findByReservald(5L) → [solicitud] → result[0].getReservald() == 5 |
| 7 | debeListarSolicitudesPorUsuario | findByUsuariold(7L) → [solicitud] → result[0].getUsuariold() == 7 |
| 8 | debeListarTodasSolicitudes | findAllByOrderByCreatedAtDesc() → [s1, s2] → result.size() == 2, result[1].precioTotal == 180 |
| 9 | debeCambiarEstadoDeSolicitud | findById(1L)=PENDIENTE → actualizarEstadoSolicitud(1L, EN_PROCESO) → response.estado == EN_PROCESO · save(solicitud) verificado |
| 10 | debeLanzarExcepcionSolicitudInexistente | findById(99L)=empty en actualizarEstadoSolicitud() → ServicioNotFoundException |

5. StripeServiceTest — 8 tests

`@Mock StripeConfig` (success/cancel URLs) · `@Mock PagoRepository` · `@Mock RabbitTemplate`
`@Mock RestTemplate` (confirmar reserva) · `@Mock StripeGateway` (abstracción sobre SDK de Stripe)

Constantes: `SESSION_ID` = "cs_test_ABC123" · `CHECKOUT_URL` = "https://checkout.stripe.com/pay/cs_test_ABC123"

Patrón StripeGateway: En lugar de `MockedStatic<Session>` (incompatible con Stripe SDK 25.x), se creó la interfaz `StripeGateway` que envuelve `Session.create()` y `Session.retrieve()`. `StripeService` depende de la interfaz — en tests se mockea normalmente como cualquier `@Mock`.

Stubs lenient en setUp(): `lenient().when(stripeConfig.getSuccessUrl())` y `getCancelUrl()` solo se usan en `crearCheckout()`, no en `confirmarPago()` → `lenient()` evita `UnnecessaryStubbingException`.

CREARCHECKOUT()

- | | |
|---|---|
| 1 debeCrearCheckoutExitosamente | stripeGateway.createSession(any()) → sessionMock → response.sessionId="cs_test_ABC123" · checkoutUrl correcto · reservald propagado |
| 2 debeLanzarExcepcionSiStripeFalla | createSession() lanza StripeException("Invalid API key") → PagoFallidoException con mensaje "Error al crear sesión" |
| 3 debeConvertirMontoCorrectamente | monto=89500.50 → redondeado HALF_UP a 89501 (long) antes de enviar a Stripe → createSession(any()) verificado |

CONFIRMARPAGO()

- | | |
|---|---|
| 4 debeConfirmarPagoExitosamente | existsByGatewayId=false → retrieveSession(SESSION_ID) → paid + amountTotal=45000 → pago COMPLETADO guardado + rabbitTemplate.convertAndSend(...,"pago.completado",...) verificado |
| 5 debeSerIdempotente | existsByGatewayId=true → retorna pago existente; retrieveSession() never · save() never · rabbitTemplate never |
| 6 debeLanzarExcepcionSiPagoNoProcesado | paymentStatus="unpaid" → PagoFallidoException "no está aprobado" · save() never |
| 7 debeLanzarExcepcionSiStripeAPIFalla | retrieveSession("cs_test_INVALID") lanza StripeException → PagoFallidoException "No se pudo verificar" · save() never |
| 8 debeContinuarSiConfirmarReservaFalla | paymentStatus="paid" + restTemplate.exchange() lanza RuntimeException → pago guardado (save verificado) · sin excepción propagada al caller |

6. UsuarioServiceTest — 8 tests

UsuarioService

StaySync_usuarios-usuarios_v1.0/.../service/UsuarioServiceTest.java

@Mock UsuarioRepository · @Mock PasswordEncoder · @Mock UsuarioMapper

Organizado con @Nested por método — patrón recomendado para servicios con muchos métodos

@NESTED REGISTRARTESTS

- | | |
|--|---|
| 1 debeRegistrarUsuario | existsByEmail=false → mapper.toEntity →
passwordEncoder.encode → save() →
response.email=="juan@test.com" |
| 2 debeLanzarExcepcionEmailDuplicado | existsByEmail("existente@test.com")==true →
EmailDuplicadoException con email en mensaje · save() never |

@NESTED OBTENERPORIDTESTS

- | | |
|--|--|
| 3 debeRetornarUsuarioActivo | findByIdAndActivoTrue(1L)=usuarioBase → mapper.toResponse
→ result.getId()==1 |
| 4 debeLanzarExcepcionNoEncontrado | findByIdAndActivoTrue(99L)=empty →
UsuarioNotFoundException |

@NESTED LISTARTESTS

- | | |
|-------------------------------------|--|
| 5 debeRetornarPaginaUsuarios | findAll(pageable)=PagelImpl([usuarioBase]) →
result.getTotalElements()==1 |
|-------------------------------------|--|

@NESTED DESACTIVARTESTS

- | | |
|---|--|
| 6 debeDesactivarUsuario | findByIdAndActivoTrue(1L) → usuarioBase.activo=false ·
save(usuarioBase) verificado |
| 7 debeLanzarExcepcionAlDesactivarInexistente | findByIdAndActivoTrue(99L)=empty →
UsuarioNotFoundException |

@NESTED ACTUALIZARTESTS

- | | |
|--------------------------------|---|
| 8 debeActualizarUsuario | findByIdAndActivoTrue(1L) →
mapper.updateEntityFromRequest(eq(usuarioBase), eq(request))
verificado con eq() · result no null |
|--------------------------------|---|

7. NotificacionServiceTest — 8 tests

NotificacionService

StaySync_Notificaciones-

notificaciones_v1.0/.../service/NotificacionServiceTest.java

@Mock NotificacionRepository · @Mock PlantillaNotificacionRepository · @Mock
JavaMailSender

Fix 1 — @Value no procesado por @InjectMocks:

ReflectionTestUtils.setField(notificacionService, "mailEnabled", true) en setUp().

Sin este fix, mailEnabled queda en false (default boolean) y el servicio no envía emails → todos los verify(mailSender).send() fallan.

Fix 2 — Java records sin setters:

UsuarioRegistradoEvent event = new UsuarioRegistradoEvent(1L, "Ana", "ana@test.com");

Los records no tienen constructor sin args ni setters — solo constructor canónico.

PROCESAREVENTO()

1	debeCrearNotificacionYEnviarEmail	findByCodigo("RESERVA_CREADA")=plantilla → renderiza vars → mailSender.send() · notificacionRepository.save() al menos 2 veces (PENDIENTE + ENVIADO)
2	debeManejarPlantillaInexistente	findByCodigo("INEXISTENTE")=empty → aun así mailSender.send() (asunto genérico) · save() al menos 2 veces

LISTENERS DE EVENTOS (RECORDS)

3	debeProcesarRegistroUsuario	UsuarioRegistradoEvent(1L,"Ana","ana@test.com") → findByCodigo("USUARIO_REGISTRO") → mailSender.send() verificado
4	debeProcesarReservaCreada	ReservaCreadaEvent(1L,1L,"juan@test.com","Juan","RES-001","101","2026-12-01","2026-12-05",400.0) → findByCodigo("RESERVA_CREADA") → send()
5	debeProcesarCambioEstado	ReservaEstadoCambiadoEvent(1L,1L,"juan@test.com","Juan","RES-001","101","CHECKIN") → findByCodigo("RESERVA_ESTADO") → send()

ENVIAREMAIL() — CICLO DE VIDA DE UNA NOTIFICACIÓN

6	debeMarcarEnviadoTrasEnvioExitoso	doNothing().when(mailSender).send() → notif.estado==ENVIADO · notif.enviadoEn != null · save(notif) verificado
7	debeMarcarFallidoTrasMáxIntentos	notif.intentos=2, maxIntentos=3 → mailSender lanza RuntimeException → estado==FALLIDO · intentos==3 · errorMsg no blank
8	noDebeMarcarFallidoSiHayIntentosRestantes	notif.intentos=0, maxIntentos=3 → falla → estado!=FALLIDO, estado!=ENVIADO · intentos==1 (primer fallo)

8. AuthServiceTest — 6 tests

AuthService

StaySync_Usuarios-usuarios_v1.0/.../service/AuthServiceTest.java

```
@Mock AuthenticationManager · @Mock UserDetailsService · @Mock JwtService
@Mock RefreshTokenRepository · @Mock UsuarioRepository · @Mock UsuarioService · @Mock
UserDetails
```

LOGIN()

- 1 **debeRetornarAuthResponseEnLoginExitoso** usuarioRepository.findByEmail → jwtService.generateToken → refreshTokenRepository.save → response: accessToken="jwt-token", refreshToken="refresh-uuid", email="admin@test.com", rol=ADMIN
- 2 **debeLanzarExcepcionConCredencialesIncorrectas** authenticationManager.authenticate() lanza BadCredentialsException → propagada al caller sin modificación

REFRESH()

- 3 **debeLanzarExcepcionConRefreshTokenInexistente** findByToken("token-invalido")==empty → TokenInvalidoException con mensaje "no encontrado"
- 4 **debeLanzarExcepcionConRefreshTokenExpirado** findByToken=expired (expiraEn = ayer) → delete(expired) verificado + TokenInvalidoException "expirado"

LOGOUT()

- 5 **debeEliminarRefreshTokenEnLogout** findByToken("valid-token")==rt → delete(rt) verificado
- 6 **noDebeLanzarExcepcionEnLogoutConTokenInexistente** findByToken("no-existe")==empty → assertThatNoException().isThrownBy(logout) — logout es idempotente

9. JwtServiceTest — 6 tests

JwtService (BFF)

StaySync_BFF-bff_v1.0/.../security/JwtServiceTest.java

Sin mocks — JwtService es una clase pura (solo opera sobre Strings y Keys)

Setup: `ReflectionTestUtils.setField(jwtService, "secret", SECRET)` — @Value no procesado por @InjectMocks

Secreto: "staysync-test-secret-key-must-be-at-least-64-characters-for-hs256-hmac-sha256" (≥64 chars para HMAC-SHA256)

Helper privado: `buildToken(email, rol, expiresInMs)` — construye JWTs reales con JJWT

1	<code>debeExtraerEmail</code>	<code>buildToken("admin@hotel.com","ADMIN",60000) → jwtService.extractEmail(token)=="admin@hotel.com"</code>
2	<code>debeExtraerRol</code>	<code>buildToken("recep@hotel.com","RECEPCIONISTA",60000) → jwtService.extractRol(token)=="RECEPCIONISTA"</code>
3	<code>tokenVigenteEsValido</code>	<code>buildToken(expiry=+60s) → isValid(token)==true</code>
4	<code>tokenExpiradoEsInvalido</code>	<code>buildToken(expiry=-1s) → isValid(token)==false (expirado en el pasado)</code>
5	<code>tokenMalformadoEsInvalido</code>	<code>isValid("esto.no.es.un.jwt")==false (JwtException capturada internamente)</code>
6	<code>stringVacioEsInvalido</code>	<code>isValid("")==false</code>

10. PagoServiceTest — 5 tests

PagoService

StaySync_Pago-pagos_v1.0/.../service/PagoServiceTest.java

@Mock PagoRepository · @Mock RabbitTemplate

setUp(): pagoBase = Pago(id=1, ref="PAY-ABCD1234", monto=200 USD, TARJETA_CREDITO, COMPLETADO)

1	debeProcesarPagoExitoso	gatewayToken="token-valido" → save() llamado 2 veces (PROCESANDO → COMPLETADO) + rabbitTemplate("pago.completado") verificado
2	debeLanzarExcepcionSinToken	gatewayToken=null → PagoFallidoException "fallido" + rabbitTemplate("pago.fallido") verificado
3	debeRetornarPagoPorId	findById(1L)=pagoBase → response.getReferencia()="PAY-ABCD1234"
4	debeLanzarExcepcionPagoNoExiste	findById(99L)=empty → PagoNotFoundException
5	debeRetornarPagosPorReserva	findByReservald(1L)=[pagoBase] → result.size()==1, result[0].getReservald()==1

11. CanalOtaServiceTest — 5 tests

CanalOtaService

StaySync_OTA-ota_v1.0/.../service/CanalOtaServiceTest.java

@Mock CanalOtaRepository · @Mock ReservaOtaRepository

setUp(): canalBooking = CanalOta("Booking.com", "BOOKING", comision=15%, activo=true)

1	debeListarActivos	findByActivoTrue()=[canalBooking] → result[0].getCodigo()="BOOKING"
2	debeRetornarCanalPorId	findById(1L)=canalBooking → result.getNombre()="Booking.com"
3	debeLanzarExcepcionNoExiste	findById(99L)=empty → CanalOtaNotFoundException
4	debeRegistrarNuevaReservaOta	findByCanalIdAndReservaldExt(1L,"BK-12345")=empty → save() llamado → result.getReservaldExt()="BK-12345"
5	debeRetornarExistenteEnDuplicado	findByCanalIdAndReservaldExt ya existe → retorna existente, save() never — idempotencia en registro OTA

12. HabitacionEventListenerTest — 3 tests

HabitacionEventListener (OTA)

StaySync_OTA-
ota_v1.0/.../messaging/HabitacionEventListenerTest.java

@Mock CanalOtaService — listener de RabbitMQ que sincroniza disponibilidad con canales externos

- | | | |
|---|--|--|
| 1 | debeProcesarEventoDisponibilidad | evento={habitacionId:10, fecha:"2025-03-01", disponible:true} → listarActivos() devuelve [canalBooking] → se itera para notificar → listarActivos() verificado |
| 2 | debeProcesarConListaCanalesVacía | evento={habitacionId:5} → listarActivos()=[] → sin error, forEach no itera · listarActivos() verificado |
| 3 | debeManejarHabitacionIdInvalidoSinExcepcion | evento={habitacionId:"no-es-numero"} → Integer.parseInt falla → catch silencioso → listarActivos() never (cortocircuito en la validación) |

13. Patrones y Técnicas Clave

Estructura AAA (Arrange — Act — Assert)

```
@Test
void debeCancelarReserva() {
    // ARRANGE — preparar datos y mocks
    reservaBase.setEstado(EstadoReserva.CONFIRMADA); // fechaEntrada en +5 días (setUp)
    when(reservaRepository.findById(1L)).thenReturn(Optional.of(reservaBase));
    when(reservaRepository.save(any())).thenReturn(reservaBase);

    // ACT — ejecutar el método bajo test
    reservaService.cancelar(1L);

    // ASSERT — estado final + efectos secundarios
    assertThat(reservaBase.getEstado()).isEqualTo(EstadoReserva.CANCELADA);
    verify(eventPublisher).publicar(eq("reserva.cancelada"), anyMap());
}
```

ReflectionTestUtils — inyectar campos @Value manualmente

```
// @InjectMocks NO procesa @Value de Spring — el campo queda en su default Java
// Solución: inyectarlo con ReflectionTestUtils en setUp()

@BeforeEach void setUp() {
    ReflectionTestUtils.setField(jwtService, "secret", "staysync-test-secret-64chars...");
    ReflectionTestUtils.setField(notificacionService, "mailEnabled", true);
}
```

Java Records — constructor canónico obligatorio

```
// INCORRECTO — records no tienen constructor sin args ni setters
UsuarioRegistradoEvent event = new UsuarioRegistradoEvent();
event.setNombre("Ana"); // ← no existe

// CORRECTO — constructor canónico (todos los campos)
UsuarioRegistradoEvent event = new UsuarioRegistradoEvent(1L, "Ana", "ana@test.com");
ReservaCreadaEvent event = new ReservaCreadaEvent(
    1L, 1L, "juan@test.com", "Juan", "RES-001", "101", "2026-12-01", "2026-12-05", 400.0);
```

lenient() — evitar UnnecessaryStubbingException

```
@BeforeEach void setUp() {
    // Sin lenient(): Mockito strict mode lanza UnnecessaryStubbingException
    // si el stub definido en @BeforeEach no se usa en TODOS los tests del archivo
    lenient().when(stripeConfig.getSuccessUrl()).thenReturn("http://localhost:3000/...");
    lenient().when(stripeConfig.getCancelUrl()).thenReturn("http://localhost:3000/...");
}
```

Fechas en tests — evitar tests flaky

```
// PROBLEMA: plusDays(1) puede fallar según la hora de ejecución
// Si el test corre a las 23:00h, mañana = solo 1 hora de margen → < 24h → falla
.fechaEntrada(LocalDate.now().plusDays(1)) // ← FLAKY

// SOLUCIÓN: suficiente margen temporal
.fechaEntrada(LocalDate.now().plusDays(5)) // ← SIEMPRE > 24h
```

StripeGateway — abstracción para testear Stripe SDK 25.x

```
// PROBLEMA: MockedStatic<Session> produce java.lang.Error: Unresolved
// con Stripe SDK 25.x (el SDK usa StripeResponseGetter internamente)

// SOLUCIÓN: interfaz inyectable
public interface StripeGateway {
    Session createSession(SessionCreateParams params) throws StripeException;
    Session retrieveSession(String sessionId) throws StripeException;
}

// En test: mock normal
@Mock private StripeGateway stripeGateway;
when(stripeGateway.createSession(any())).thenReturn(sessionMock);

// En producción: StripeGatewayImpl delega al SDK
return Session.create(params); // ← única llamada real al SDK
```

@MockitoBean — reemplazante de @MockBean en Spring Boot 3.4+

```
// DEPRECADO (Spring Boot 3.4+)
import org.springframework.boot.test.mock.mockito.MockBean;
@MockBean private ReservaService reservaService;

// CORRECTO (reemplazante oficial desde Spring Boot 3.4)
import org.springframework.test.context.bean.override.mockito.MockitoBean;
@MockitoBean private ReservaService reservaService;
```

14. Cómo Ejecutar los Tests Unitarios

Comando	Qué hace
<code>mvn test</code>	Todos los tests del módulo (unitarios + integración)
<code>mvn test -Dtest="*ServiceTest"</code>	Solo tests de servicios unitarios
<code>mvn test -Dtest="*ListenerTest"</code>	Solo tests de listeners
<code>mvn test -Dtest="ReservaServiceTest#debeCancelarReserva"</code>	Un test específico
<code>mvn verify</code>	Tests + reporte JaCoCo (target/site/jacoco/index.html)
<code>mvn test -pl StaySync_Reservas-reservas_v1.1</code>	Solo el módulo de reservas